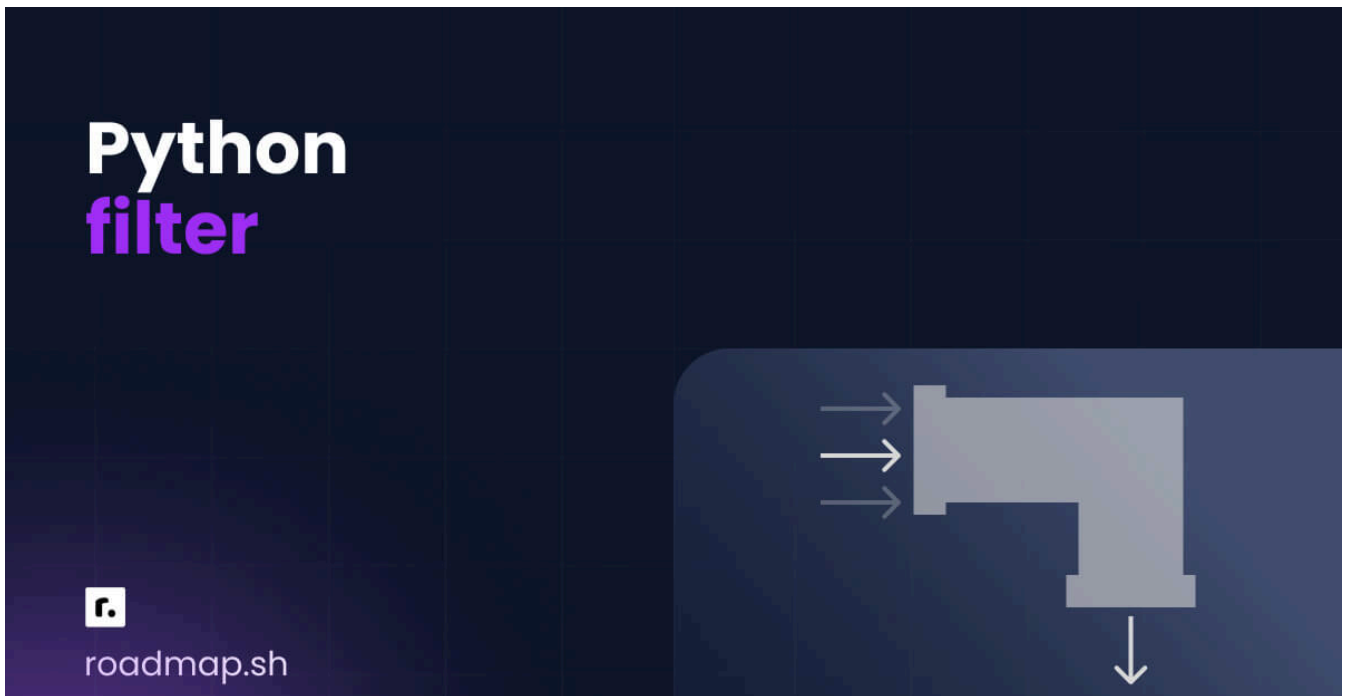


Master Python Filter: синтаксис, примеры и лучшие практики

 Кимберли Фессел Предпочесть нас в Google ☆

В условиях современного изобилия информации в интернете как никогда важно уметь отделять сигнал от шума. Так как же создать и обеспечить соблюдение этого VIP-списка, чтобы в него попадали только качественные данные? Встроенная функция `filter()` в Python действует как вышибала, проверяя каждый элемент на входе и пропуская только те, которые соответствуют **True** в вашем условии.

Содержание



Что такое `filter()` в Python?

Синтаксис и Возвращаемое значение



Основные примеры

`filter()` в сравнении с генераторами списков и генераторными выражениями

Сочетание `filter()` с `map()` и `reduce()`

Фильтрация за пределами списков в анализе данных

Распространенные Подводные Камни

Подведение Итогов

В этой статье мы расскажем, что такое `filter()` и как он работает. Вы увидите множество простых и более сложных примеров его использования, а также мы сравним его с другими вариантами фильтрации в Python, такими как списковые включения. В конце статьи вы также узнаете о нескольких распространенных ошибках, чтобы `filter()` избежать их в своем коде.

Что такое `filter()` в Python?

`filter()` — это встроенная функция Python для выбора элементов из итерируемого объекта, то есть любого объекта, который можно перебрать, на основе заданного вами условия. Она применяет одно и то же условие к каждому элементу и оставляет только те, которые ему соответствуют. Вы часто будете использовать `filter()` для очистки данных и других задач по их отбору.

Синтаксис и возвращаемое значение

При использовании `filter()` необходимо указать два обязательных параметра: функцию условия и итерируемый объект. Синтаксис `filter()` выглядит следующим образом:

```
filter(function, iterable)
```

Функция условия должна возвращать **True** или **False** для каждого элемента итерируемого множества, а `filter()` сохраняет только те элементы, которые возвращают **True**.

Функция `filter()` возвращает объект фильтра. Это ленивый итератор. Это значит, что Python не проверяет каждый элемент итерируемого объекта сразу. Вместо этого он оценивает элементы по одному, только когда вы запрашиваете их. Фильтрация не происходит до тех пор, пока вы не переберете возвращаемый объект. Вы можете пройти по нему циклом, преобразовать его в список или передать другой функции, которая принимает итератор.

Также имейте в виду, что `filter()` не изменяет исходный итерируемый объект. Он создает новый объект, который выдает только те элементы, которые удовлетворяют условию.

✦ Преподаватель искусственного Интеллекта

Опишите ленивую оценку в Python и ее отличия от нетерпеливой оценки.



Basic Examples

Now that you've learned the basics of the `filter()` function, it's time to see it in action. Here are several neat examples to demonstrate practical uses for `filter()`.

Using Python's `filter()` with a Custom Function

`filter()` expects two inputs: your condition function and the iterable to apply it to. You can write your own custom function for `filter()` as long

as your function returns **True** or **False** for each item.

Say you have a list of words and only want to keep short words with five or fewer characters. Your function checks if a word has five or fewer characters, and `filter()` uses the results to narrow down your list.

python



```
def is_short_word(word):
    return len(word) <= 5

words = ["computer", "apple", "dog", "ceiling", "mail"]

short_words = filter(is_short_word, words)

print(list(short_words))

# Expected result:
# ['apple', 'dog', 'mail']
```

Notice that we converted `short_words` to a list before printing the results.

Lambda Functions

If your condition function proves especially simple, you can write it as a one-line lambda function direction within `filter()` instead of defining a separate function. Lambdas act just like regular user-defined functions, but they're anonymous and concise, making them a convenient choice for short, one-off conditions.

The previous example of finding short words can be rewritten with `lambda` instead of the custom `is_short_word()` function:

python



```
words = ["computer", "apple", "dog", "ceiling", "mail"]
```

```
short_words = filter(lambda x: len(x) <= 5, words)

print(list(short_words))

# Expected result:
# ['apple', 'dog', 'mail']
```

You will commonly see user-defined functions as well as lambdas as the function argument passed to `filter()`. In general, lambdas work well for simple conditions, while named functions are typically easier to read and reuse for more complex logic.

Even Numbers Example

The `filter()` function works for collections of numbers just as well as strings. One classic example of `filter()` selects only the even numbers from a list of values:

```
python

numbers = [0, 1, 2, 3, 4, 5, 6]

evens = filter(lambda x: x % 2 == 0, numbers)

print(list(evens))

# Expected result:
# [0, 2, 4, 6]
```

Remove Empty Strings and Falsy Values

Truthiness is a Python concept that extends the idea of Booleans beyond just **True** and **False**. Values are considered either “truthy” or “falsy.”

Common falsy values include:

- **False**
- **None**

- **0** or **0.0**
- **"** (empty string)
- **[]**, **{}**, **()**, or **set()** (empty collections)

You can capitalize on this inherent truthiness with the `filter()` function. Remember that `filter()` only keeps values where the condition function evaluates to **True**. If you pass **None** as the condition function, `filter()` uses the identity of each item itself as the test, keeping only the truthy values.

You can remove empty strings from a list of names like this:

```
python

names = ["Stacey", "Yasmin", "", "", "Todd", "", "Raj"]

valid_names = filter(None, names)

print(list(valid_names))

# Expected result:
# ['Stacey', 'Yasmin', 'Todd', 'Raj']
```

Here, `filter()` checks the truthiness of each name and excludes the empty strings.



AI Tutor

Provide situations where `filter()` relies on truthiness in Python.



`filter()` vs. List Comprehensions vs. Generator Expressions

The `filter()` function isn't the only filtering mechanism in Python. List comprehensions and generator expressions accomplish similar tasks, but

with different tradeoffs in readability, flexibility, and memory usage.

This is what the even number example looks like for each method:

```
python

numbers = [0, 1, 2, 3, 4, 5, 6]

# filter()
evens_filter = filter(lambda x: x % 2 == 0, numbers)

# comprehension
evens_list_comp = [x for x in numbers if x % 2 == 0]

# generator
evens_generator_exp = (x for x in numbers if x % 2 == 0)

print(list(evens_filter))
print(evens_list_comp)
print(list(evens_generator_exp))

# Expected result:
# [0, 2, 4, 6]
# [0, 2, 4, 6]
# [0, 2, 4, 6]
```

Each technique ultimately gives the same values. Notice that the comprehension already returns a `list`, but both `filter()` and the generator produce lazy iterators that we must consume (say, with `list()`) to display all their values.

Here's a more detailed breakdown comparing each method:

	<code>filter()</code>	List Comprehension	Generator Expression
Best suited for	Simple functions	Pythonic syntax	Data pipelines
Returns	filter object	list	Generator

	<code>filter()</code>	List Comprehension	Generator Expression
Lazy	Yes	No	Yes
Requires function	Yes	No	No
Multiple conditions	Less readable	Readable	Readable
Memory efficient	Yes	No	Yes

Practically, you'll likely reach for `filter()` if you already have your condition function defined or want to work lazily. Comprehensions tend to show up more in daily work since they're easy to create and understand. Generators use syntax similar to comprehensions, but they evaluate lazily, making them a great choice for large datasets or streaming data.

Is the Filter Function Actually Faster?

For many real-world problems, the performance difference between `filter()`, list comprehensions, and generators is small. Comprehensions are often just as fast or even slightly faster than `filter()`; however, they evaluate immediately, which can lead to memory inefficiencies for large datasets.

Rather than choosing `filter()` for speed, pick the approach that makes your code the cleanest. If you already have a condition function defined, you can easily pop it into `filter()` to apply it to an iterable. Otherwise, list comprehensions are generally the Pythonic choice for straightforward filtering.

Combining `filter()` with `map()` and `reduce()`

`filter()` , `map()` , and `reduce()` often come as a trio because they're fundamental to functional programming in Python and other languages. Each function has a distinct purpose:

- **`filter()`** selects items
- **`map()`** transforms items
- **`reduce()`** aggregates items into a single value

Suppose you'd like to select all even numbers, square them, and take the sum of their squares. After importing `reduce()` from `functools` , which is part of Python's standard library, this code combines `filter()` , `map()` , and `reduce()` into one unified pipeline:

```
python

from functools import reduce

numbers = [0, 1, 2, 3, 4, 5, 6]

evens = filter(lambda x: x % 2 == 0, numbers) # [0, 2, 4, 6]
squares = map(lambda x: x**2, evens) # [0, 4, 16, 36]
total = reduce(lambda x, y: x + y, squares) # 56

print(total)

# Expected result:
# 56
```

While you'll often see `filter()` , `map()` , and `reduce()` taught together, modern Python code more heavily relies on comprehensions and built-in functions like `sum()` over `map()` and `reduce()` . Nonetheless, it's good to know how these components can work together, especially if reading functional-style code.

Filtering beyond Lists in Data Analysis

The article demonstrations thus far have focused on filtering Python lists, but you can use `filter()` for many other data structures as well. Data analysts and data scientists frequently rely on generators, file objects, dictionaries, and other iterable objects; `filter()` can operate on all of these.

Say you have a generator of values ranging from 0 to 99. The following code filters to only large numbers strictly greater than 95:

```
python

my_generator = (x for x in range(100))

high_values = filter(lambda x: x > 95, my_generator)

for value in high_values:
    print(value)

# Expected result:
# 96
# 97
# 98
# 99
```

In this case, we used a loop to consume the resulting iterator and print each value. We could have just as easily converted `high_values` to a list, as in previous examples.

Data professionals frequently need to read data from flat files, which may have extraneous lines not applicable to the analysis. You may use `filter()` to skip these lines when processing data from your file:

```
python

with open("sales.csv") as f:
```

```
valid_rows = filter(is_valid_row, f)

for row in valid_rows:
    process_row(row)
```

Here, `is_valid_row()` and `process_row()` represent user-defined functions. The `is_valid_row()` function may check for valid characters, reasonable values, or any other test needed to verify that a row is valid. Likewise, the `process_row()` function may transform, clean, analyze, or aggregate information as necessary.

`filter()` is not limited to lists when analyzing data. As long as your object is iterable, you can apply the same filtering logic to process only items relevant to your task. However, there are a few `filter()` behaviors that often surprise new Python developers. We cover common “gotchas” in the next section to help you avoid `filter()` issues.



AI Tutor

Explain how data analysts use `filter()` beyond working with lists.



Common Pitfalls

Python’s `filter()` function selects items based on a condition function in a straightforward way, but there are a few common sources of errors when working with `filter()`. Review these top pitfalls to ensure your `filter()` code stays bug-free.

Iterator Exhaustion

`filter` objects are iterators. That means you can only fully consume them once. Attempting to do so again results in no data, which often surprises those new to iterators.

Here's code where we look for people whose names start with the letter "K":

python



```
people = ["Kamila", "Ken", "Nola", "Monika", "Kristen"]

k_names = filter(lambda name: name[0].upper() == "K", people)

print(list(k_names))
print(list(k_names))

# Expected result:
# ['Kamila', 'Ken', 'Kristen']
# []
```

Those with "K" names appear as expected with the first `print()` statement. When we try to convert the `k_names` to a list for the second time, it's already exhausted, and we print an empty list. This happens because `filter()` does not store the filtered results; it generates each value only when requested. Once you consume those values, they are no longer available.

Be sure to store the results if you need to use them multiple times. After consuming the return object, you would need to recreate it to iterate over the values again.



Show me a few cases of iterator exhaustion for `filter()`.



Forgetting to Wrap in `list()`

The `filter()` function returns an iterator, not a list. You'll need to consume that resulting object by converting it to a list, looping over it, or otherwise passing it to a function that accepts an iterable to view your results.

If you attempt to directly print `filter()` 's return object, you see the object representation, not the values themselves.

python



```
numbers = [0, 1, 2, 3, 4, 5, 6]

evens = filter(lambda x: x%2 == 0, numbers)

print(evens)

# Expected result:
# <filter object at 0x103438430>
```

Remember to wrap the resulting object in a `list()` if you need to inspect or store all the values. Alternatively, iterate over it directly to process items one at a time and to avoid storing everything in memory when working with especially large datasets.

Using Complicated Lambdas

Lambda functions are great for short, simple functions. Long or otherwise complicated lambdas prove difficult to read and debug.

Try to avoid the following to keep your lambdas concise:

- Multiple lengthy conditions (`and` , `or`)
- Nested logic
- Complex string manipulation
- Type conversions
- Error handling
- Side calculations unrelated to the filter condition

If the lambda of your `filter()` function requires any of the features above, consider rewriting it as a separate, named user-defined function. There, you can create a function that has multiple lines, comments, and in-depth logic that you can even use elsewhere in your code. Named functions are typically easier to debug because you can print intermediate values and test the function independently of `filter()`.

Wrapping Up

Python's `filter()` function lets you filter a list or other iterable by testing each item against a condition function. Call it using `filter(function, iterable)` syntax, where function may be user-defined or a lambda. Unlike a list comprehension, `filter()` evaluates lazily. Its resulting iterator does not store values; instead, it generates them on demand, such as when you convert the results to a list.

Try out the AI Tutor to `filter()` through your knowledge. This starting prompt will put you on the right track:



AI Tutor

Quiz me on Python's `filter()` function including its lazy evaluation.



Related Guides



[Python `reduce\(\)`: The Complete Guide \(With Examples\)](#)

[Python `Max Int`: Understanding Arbitrary Precision Integers](#)

[Python `KeyError` Exceptions: Causes and Fixes Explained](#)

[Python `Null \(None\)`: Guide to Missing Values and `NoneType`](#)

[Python Backend Development: Build Your First API](#)

[Python Backend Frameworks: How to Choose the Right One](#)

[Python Return Multiple Values: 4 Methods & Examples](#)

Python Multiline Strings: The Complete Guide

Python not Operator: The Complete Guide to Logical Negation

Python Print New Line: Methods, Examples, and Best Practices

Join the Community

roadmap.sh is the 6th most starred project on GitHub and is visited by hundreds of thousands of developers every month.

Rank **out of 28M!**

364K

GitHub Stars

★ **Star us on GitHub**
Help us reach #1

+90k **every month**

+2.8M

Registered Users

👤 **Register yourself**
Commit to your growth

+2k **every month**

50K

Discord Members

🗨️ **Join on Discord**
Join the community

[Roadmaps](#) [Guides](#) [FAQs](#) [YouTube](#)

 **roadmap.sh** by [@nilbuild](#)

Community created roadmaps, best practices, projects, articles, resources and journeys to help you choose your path and grow in your career.

© roadmap.sh · [Terms](#) · [Privacy](#) ·    



The top DevOps resource for Kubernetes, cloud-native computing, and large-scale development and deployment.

DevOps · Kubernetes · Cloud-Native

 [Cookie Settings](#)